UNIVERSITAS NEGERI SURABAYA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PRODI S1 SAINS DATA

NASKAH SOAL UJIAN TENGAH SEMESTER
Matakuliah : Graf Sains Data
SKS : 3
Tanggal : Minggu ke-8/9, Gasal 2026/2027
Dosen : TIM

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

Materi : Python Driver untuk (Neo4J) dan Rekomendasi
Sifat : Individu

Petunjuk Umum

1. Ikuti langkah-langkah aktivitas individu sesuai dengan petunjuk.
2. Perhatikan poin-poin utama dalam tiap pertanyaan.

1. Tugas Pendahuluan

1.1 Persiapan dan Perkenalan Lingkungan

1. Buatlah sebuah environment baru menggunakan venv atau uv dan jalankan perintah seperti berikut $

   ```
   python -m venv myenv
   ```

   atau, untuk uv seperti

   ```
   $ uv venv myvenv --python 3.10
   ```

   Pembuatan virtual environment ini sangat disarankan untuk proses isolasi development environment sehingga dapat meminimalisasi ”disrupsi” paket-paket dependensi dari *local projects* di lingkungan kerja (laptop / PC) masing-masing.

2. Matikan paket virtual environment yang lain (jika ada).

   ```
   $ conda deactivate
   ```

   Aktifkan virtual environment baru myvenv

   ```
   $ source myvenv/bin/activate
   ```

3. Install dependensi paket-paket seperti yang ada di dalam file requirements.txt seperti berikut

```
neo4j>=4.0,<4.3
imdbpy==2020.9.25
nose==1.3.7
cinemagoerng>=0.4
imdbinfo>=0.5
imdbpy>=2020.9.25
neo4j==4.2.1
```

   dengan perintah kurang lebih sebagai berikut:

   ```
   uv pip install -r requirements.txt
   ```

FAKULTAS MATEMATIKA
DAN ILMU PENGETAHUAN ALAM
PRODI S1 SAINS DATA

1.2 Mempelajari Penggunaan Driver Neo4J

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

1. Ikuti tutorial penggunaan driver Neo4J untuk melakukan pembuatan objek dan akses objek berikut. 2. Baris berikut mendefinisikan string koneksi ke database Neo4J:

```
from neo4j import GraphDatabase, RoutingControl

URI = "neo4j://localhost:7687"
AUTH = ("neo4j", "password")
```

Ganti informasi *credential* sesuai dengan kebutuhan.

3. Baris perintah berikut melakukan pembuatan sebuah Node berikut dengan relationship-nya.

```
def add_friend(driver, name, friend_name):
driver.execute_query(
"MERGE (a:Person {name: $name}) "
"MERGE (friend:Person {name: $friend_name}) "
"MERGE (a)-[:KNOWS]->(friend)",
name=name, friend_name=friend_name, database_="neo4j",
)
```

Perhatikan perintah execute query.

4. Untuk mencetak dan menjalankan keseluruhan

```
def print_friends(driver, name):
records, _, _ = driver.execute_query(
"MATCH (a:Person)-[:KNOWS]->(friend) WHERE a.name = $name "
"RETURN friend.name ORDER BY friend.name",
name=name, database_="neo4j", routing_=RoutingControl.READ,
)
for record in records:
print(record["friend.name"])

with GraphDatabase.driver(URI, auth=AUTH, ) as driver:
add_friend(driver, "Arthur", "Guinevere")
add_friend(driver, "Arthur", "Lancelot")
add_friend(driver, "Arthur", "Merlin")
print_friends(driver, "Arthur")
```

1. Apa nama database yang digunakan pada tutorial ini?
2. Sebutkan akses lain (seperti sintaks DDL/DML CYPHER) yang dapat digunakan mengakses database lain dalam 1 instance!
3. Apa jenis graf pertemanan yang telah dibuat di sini?

Jalankan perintah berikut untuk melakukan *truncate* pada konten database.

```
MATCH (n) DETACH DELETE n;
```

→ Hati-hati dalam menggunakan perintah ini pada *shared database*.

1.3 Melakukan proses Impor Data dari RetailRocket ke Database Neo4J Sumber data:

Roman Zykov

1. Unduh data dari Kaggle dengan perintah berikut:

(a) Windows PowerShell:

curl.exe -L -o ecommerce-dataset.zip `
ω→https://www.kaggle.com/api/v1/datasets/download/retailrocket/ecommerce-dataset (b) Unix-based (Linux / MacOS):

curl -L -o ecommerce-dataset.zip \
ω→https://www.kaggle.com/api/v1/datasets/download/retailrocket/ecommerce-dataset 2. Perhatikan fitur yang telah diunduh dengan rincian sebagai berikut:

(a) timestamp: Waktu dalam format unix epoch.

(b) visitorid: ID unik pengunjung.

(c) event: Jenis interaksi *default: view, addtocart, transaction*.

(d) itemid: ID unik produk.

(e) transactionid: ID transaksi, hanya terisi saat event berjenis 'transaction'.

3. Buatlah mekanisme pembuatan session Neo4J baru dengan perintah kurang lebih seperti berikut (sumber: Neo4J Driver 6.4).

```
from neo4j import GraphDatabase

driver = GraphDatabase.driver(uri, auth=(user, password))
session = driver.session(database="system")
```

atau, modifikasi dengan argparse, contohnya:

```
def parse_arguments():
parser = argparse.ArgumentParser(
description="Impor dataset RetailRocket (events.csv) ke database Neo4J." )
parser.add_argument(
"-u", "--user", default="neo4j", help="Username Neo4j (default: neo4j)" )
parser.add_argument(
"-p",
"--password",
default="neo4j",
help="Password Neo4J (default: neo4j)",
)
parser.add_argument(
"-b",
"--bolt",
default="neo4j://127.0.0.1:7687",
help="Neo4J URI / Bolt (default: neo4j://127.0.0.1:7687)",
)
parser.add_argument(
"-s",
"--source",
default="data",
help="Direktori/folder sumber data (default: data)",
)
return parser.parse_args()
```


Diawali oleh pemeriksaan keberadaan berkas events.csv

```
csv_file_path = os.path.join(args.source, "events.csv")
if not os.path.exists(csv_file_path):
print(f"Error: File '{csv_file_path}' tidak ditemukan!")
sys.exit(1)
```

Dilanjutkan dengan pembuatan koneksi

```
try:
driver = GraphDatabase.driver(
args.bolt, auth=(args.user, args.password) )
```

```
driver.verify_connectivity()
except Exception as e:
print(f"Gagal terhubung ke database: {e}") sys.exit(1)
```

Sesuaikan dengan kebutuhan Anda.

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

4. Anda lanjutkan dengan proses impor dengan mekanisme *batch*, kurang lebih sebagai berikut:

```
batch_size =

with driver.session() as session:
create_constraints(session)

print(f"Membaca file data dari: {csv_file_path}")
chunks = pd.read_csv(csv_file_path, chunksize=batch_size)

total_rows =
total_chunks = (total_rows // batch_size) + 1

print("Memulai proses impor data ke Neo4j...")
for chunk in tqdm(
chunks, total=total_chunks, desc="Mengimpor Batch Data"
):
# Membersihkan data null/NaN pada transactionid
chunk["transactionid"]=(
chunk["transactionid"].fillna("").astype(str)
)

records = chunk.to_dict(orient="records")
session.execute_write(import_batch, records)

driver.close()
print("Proses impor berhasil.")
```

5. Selanjutnya, Anda membutuhkan baris perintah untuk mengimpor data dalam CYPHER, contohnya:

```
def import_batch(tx, batch):
cypher_query = """
UNWIND $batch AS row
MERGE (v:Visitor {visitorId: row.visitorid})
MERGE (i:Item {itemId: row.itemid})

WITH v, i, row
```

UNIVERSITAS NEGERI SURABAYA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PRODI S1 SAINS DATA

```
CALL {
WITH v, i, row
WITH v, i, row WHERE row.event = 'view'
CREATE (v)-[:VIEWED {timestamp: row.timestamp}]->(i)

UNION ALL

WITH v, i, row
WITH v, i, row WHERE row.event = 'addtocart' CREATE (v)-[:ADDED_TO_CART {timestamp: row.timestamp}]->(i)
UNION ALL

WITH v, i, row
WITH v, i, row WHERE row.event = 'transaction'
CREATE (v)-[:BOUGHT {timestamp: row.timestamp, transactionId: row.transactionid}]->(i) }
"""
tx.run(cypher_query, batch=batch)
```

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

2. Soal UTS

Petunjuk Khusus

1. Deadline: Selasa, 20/10 23:59.
2. Modifikasi kode sumber sesuai dengan kebutuhan Anda.
3. Unggah pekerjaan Anda (dokumentasi notebook ipynb dan penjelasan, serta dataset) pada akun Github Anda, dan tuliskan URL github Anda pada Google Classroom.
4. Sesi kelas pada jadwal masing-masing akan menanya beberapa mahasiswa secara acak.

2.1 Wrangling dan EDA (Maks. skor: 45)

1. Tentukan 2 periode waktu sebagai filter dari data interaksi. Kombinasi periode yg dipilih per individu tidak boleh sama dalam 1 kelas.
   Misal: Fold training (P1) → 01/2023 - 03/2023; Fold testing (P2) 01/2024 - 06/2024. 2. Terapkan proses filter

ini pada timestamp pada proses persiapan data, lalu simpan pada sebuah file .csv!

3. Lakukan query untuk mendapatkan komposisi dari *subgraph* yang dibuat. Uraikan jenis simpul dan jumlah masing-masing simpul dengan Python API! Jelaskan CYPHER SQL yang digunakan!
4. Lakukan Vector Space Modelling (VSM) terhadap simpul yang diasumsikan sebagai User. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!
5. Lakukan VSM terhadap simpul yang diasumsikan sebagai Item. Uraikan bagaimana karakteristiknya (di mensi,

shape, plot networkx, dst.)!

6. Berdasarkan eksplorasi no. 4 dan 5, uraikan proyeksi manakah yang lebih baik (efektif/efisien)!
7. Berdasarkan analisis no. 6, lakukan perhitungan similarity pada hasil graf yg terproyeksi dengan menggu nakan Python

UNIVERSITAS NEGERI SURABAYA FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN
ALAM PRODI S1 SAINS DATA
Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

   (a) Tuliskan CYPHER SQL untuk mendapatkan 5 records dengan similarity tertinggi (Anda tentukan sendiri record acuannya)!
   (b) Tuliskan kembali nilai similarity-similarity tersebut pada graf yang telah ada! Perlihatkan hasil query nya!
8. Tuliskan CYPHER SQL dan tunjukkan hasil graf-nya untuk memperlihatkan kondisi: sebelum dan sesudah perhitungan similarity!

2.2 Analytics (Maks. skor: 55)

1. Pilih seorang *user* secara acak (tentukan proses acak-nya, dan harus bisa di-reproduksi), lalu
   (a) tuliskan skrip Python dalam merekomendasikan KNN-nya!
   (b) berikan 3 item yang diiklankan untuk k = 5 pada penentuan KNN nya!
2. Berdasarkan hasil rekomendasi Anda no. 1b Analytics, hitunglah:
   (a) Hit@5
   (b) NDCG@5

— Semoga Sukses! —

Kesimpulan

1. Pertanyaan Pendahuluan (Checkpint)

- Apa nama database yang digunakan pada tutorial ini?

  Jawab : neo4j

- Sebutkan akses lain (seperti sintaks DDL/DML CYPHER) yang dapat digunakan mengakses database lain dalam 1 instance!

  Jawab : DDL misalnya CREATE CONSTRAINT , DML misalnya CREATE untuk membuat node atau relasi, MERGE untuk membuat data jika belum tersedia, MATCH untuk mencari data, SET untuk memperbarui properti, dan DELETE untuk menghapus data.

- Apa jenis graf pertemanan yang telah dibuat di sini?

  Jawab : Directed graph

2. Wrangling dan EDA (Maks. skor: 45)

- Tentukan 2 periode waktu sebagai filter dari data interaksi. Kombinasi periode yg dipilih per individu tidak boleh sama dalam 1 kelas.

  Jawab : Dataset Retailrocket dibagi menjadi dua periode waktu agar model dibangun menggunakan data historis dan hasilnya dapat dianalisis pada periode yang berbeda. Pembagian dilakukan berdasarkan kolom timestamp, yang menggunakan format Unix timestamp dalam milidetik.

  - P1 (Training): 3 Mei 2015 – 24 Juli 2015.
  - P2 (Testing): 25 Juli 2015 – 18 September 2015.

- Terapkan proses filter ini pada timestamp pada proses persiapan data, lalu simpan pada sebuah file .csv!

  Jawab :

```
import pandas as pd
from pathlib import Path

# Lokasi dataset asli
CSV_PATH = Path(
    r"C:\Users\LENOVO\Documents\UTS GSD\ecommerce-dataset\events.csv"
)

# Folder penyimpanan hasil
OUTPUT_DIR = Path(r"C:\Users\LENOVO\Documents\UTS GSD")

# Baca dataset asli
df = pd.read_csv(CSV_PATH)

# Konversi Unix timestamp milidetik ke waktu UTC
df["datetime"] = pd.to_datetime(
    df["timestamp"], unit="ms", utc=True
)

# Filter P1: 3 Mei–24 Juli 2015
```

```
train_df = df[
    (df["datetime"] >= "2015-05-03 00:00:00+00:00") &
    (df["datetime"] <  "2015-07-25 00:00:00+00:00")
].copy()

# Filter P2: 25 Juli–18 September 2015
test_df = df[
    (df["datetime"] >= "2015-07-25 00:00:00+00:00") &
    (df["datetime"] <  "2015-09-19 00:00:00+00:00")
].copy()

# Hapus kolom datetime tambahan; timestamp asli tetap dipertahankan
train_df.drop(columns=["datetime"], inplace=True)
test_df.drop(columns=["datetime"], inplace=True)

# Simpan sebagai file baru
train_path = OUTPUT_DIR / "retailrocket_train_new.csv"
test_path = OUTPUT_DIR / "retailrocket_test_new.csv"

train_df.to_csv(train_path, index=False)
test_df.to_csv(test_path, index=False)

# Tampilkan ringkasan hasil
print("P1 Training:", f"{len(train_df):,}", "baris")
print("P2 Testing:", f"{len(test_df):,}", "baris")
print("\nKolom:", train_df.columns.tolist())
print("File training:", train_path)
print("File testing:", test_path)
print("\nEkspor CSV baru selesai.")
```

1. Lakukan query untuk mendapatkan komposisi dari subgraph yang dibuat. Uraikan jenis simpul dan jumlah masing-masing simpul dengan Python API! Jelaskan CYPHER SQL yang digunakan!
   Jawab :

```
# SUBGRAF BIPARTIT VISITOR-ITEM

# 1. Pasangan Visitor-Item unik dari data Training
pairs = train_df[["visitorid", "itemid"]].drop_duplicates()

# 2. Hitung jumlah visitor unik untuk setiap item
item_degree = pairs.groupby("itemid")["visitorid"].nunique()

# 3. Pilih maksimal 40 item dengan minimal 3 visitor
selected_items = (
    item_degree[item_degree >= 3]
    .sort_values(ascending=False)
    .head(40)
    .index
```

```
)

# 4. Ambil hubungan visitor-item pada item terpilih
sub_pairs = pairs[pairs["itemid"].isin(selected_items)].copy()

# 5. Batasi maksimal 60 visitor dengan interaksi terbanyak
selected_visitors = (
    sub_pairs["visitorid"]
    .value_counts()
    .head(60)
    .index
)

sub_pairs = sub_pairs[
    sub_pairs["visitorid"].isin(selected_visitors)
].copy()

# 6. Bentuk graf bipartit
G_sub = nx.Graph()

for row in sub_pairs.itertuples(index=False):
    visitor = f"V_{row.visitorid}"
    item = f"I_{row.itemid}"

    G_sub.add_node(visitor, node_type="Visitor", bipartite=0)
    G_sub.add_node(item, node_type="Item", bipartite=1)
    G_sub.add_edge(visitor, item)

# 7. Pisahkan simpul berdasarkan jenisnya
visitor_nodes = [
    n for n, d in G_sub.nodes(data=True)
    if d["node_type"] == "Visitor"
]

item_nodes = [
    n for n, d in G_sub.nodes(data=True)
    if d["node_type"] == "Item"
]

# 8. Posisi simpul
pos = nx.spring_layout(
    G_sub,
    seed=42,
    k=0.6,
    iterations=100
)

# 9. Visualisasi
```

```
fig, ax = plt.subplots(figsize=(14, 10))

nx.draw_networkx_edges(
    G_sub, pos,
    ax=ax,
    edge_color="gray",
    alpha=0.3,
    width=0.8
)

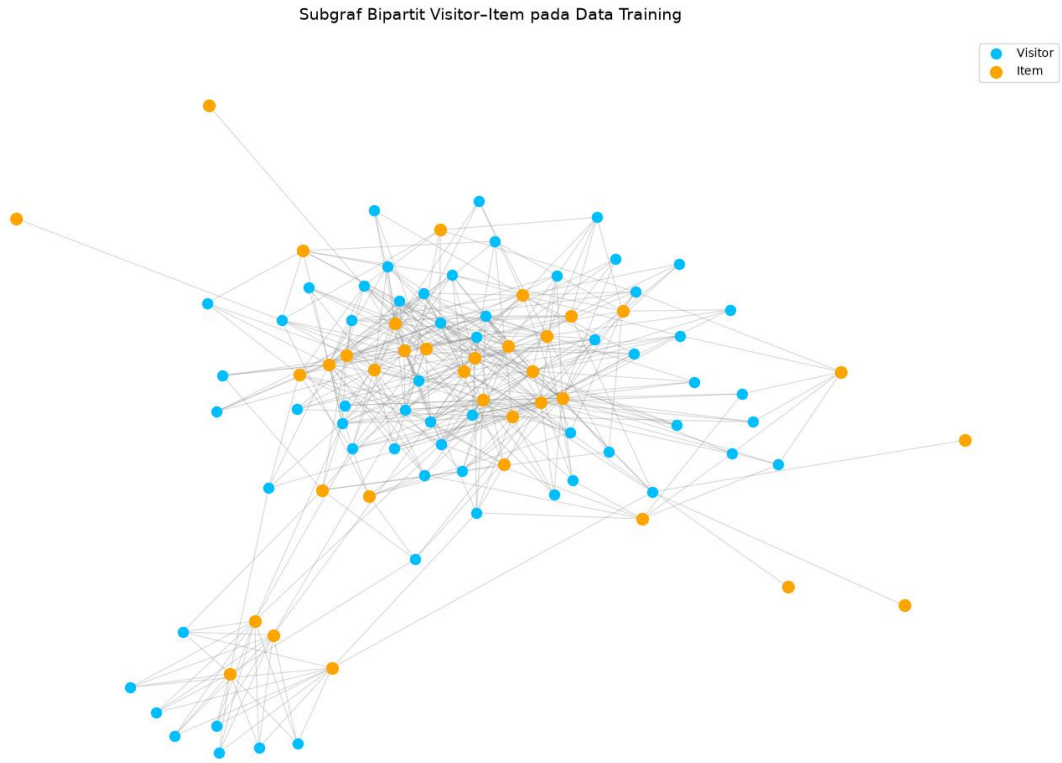
nx.draw_networkx_nodes(
    G_sub, pos,
    nodelist=visitor_nodes,
    node_color="deepskyblue",
    node_size=75,
    label="Visitor",
    ax=ax
)

nx.draw_networkx_nodes(
    G_sub, pos,
    nodelist=item_nodes,
    node_color="orange",
    node_size=100,
    label="Item",
    ax=ax
)

ax.set_title(
    "Subgraf Bipartit Visitor-Item pada Data Training",
    fontsize=14,
    pad=15
)
ax.legend()
ax.axis("off")
fig.tight_layout()
plt.show()

# 10. Karakteristik subgraf
print("=== KARAKTERISTIK SUBGRAF ===")
print("Jumlah Visitor:", len(visitor_nodes))
print("Jumlah Item:", len(item_nodes))
print("Jumlah simpul:", G_sub.number_of_nodes())
print("Jumlah sisi:", G_sub.number_of_edges())
print("Graf bipartit:", nx.is_bipartite(G_sub))
print("Komponen terhubung:", nx.number_connected_components(G_sub))
print("Item awal terpilih:", len(selected_items))
print("Pasangan interaksi dalam subgraf:", len(sub_pairs))
```

```
=== KARAKTERISTIK SUBGRAF ===
Jumlah Visitor: 60
Jumlah Item: 35
Jumlah simpul: 95
Jumlah sisi: 404
Graf bipartit: True
Komponen terhubung: 1
Item awal terpilih: 40
                    Pasangan interaksi dalam subgraf: 404
```



- Interpretasi :
  Visualisasi graf bipartit dibentuk dari data training RetailRocket dengan memilih 40 item yang memiliki interaksi dengan minimal tiga visitor, kemudian membatasi jaringan pada 60 visitor dengan interaksi terbanyak. Setelah proses penyaringan, terbentuk subgraf yang terdiri atas 60 simpul Visitor dan 35 simpul Item, sehingga totalnya 95 simpul dengan 404 sisi. Hasil pemeriksaan menunjukkan bahwa graf bersifat bipartit dan memiliki satu komponen terhubung. Perbedaan antara 40 item yang dipilih pada tahap awal dan 35 item yang muncul pada graf disebabkan oleh penyaringan visitor, sehingga beberapa item tidak lagi memiliki hubungan pada subgraf akhir. Visualisasi ini memperlihatkan pola hubungan antara pengguna dan produk secara lebih terfokus.

- **Menghitung simpul Visitor**

```
MATCH (v:Visitor)
RETURN count(DISTINCT v) AS jumlah_visitor;
```

Penjelasan:

- `MATCH (v:Visitor)` digunakan untuk mencari seluruh node yang memiliki label `Visitor` pada database Neo4j. Variabel `v` mewakili setiap node Visitor yang ditemukan.
- `RETURN` digunakan untuk menampilkan hasil query.
- `count(DISTINCT v)` menghitung jumlah node Visitor yang berbeda. Penggunaan `DISTINCT` memastikan setiap node dihitung satu kali.
- `AS jumlah_visitor` memberikan nama kolom hasil menjadi `jumlah_visitor` agar lebih mudah dibaca.

➢ **Menghitung simpul Item**

```
MATCH (i:Item)
RETURN count(DISTINCT i) AS jumlah_item;
```

Penjelasan:

- `MATCH (i:Item)` mencari seluruh node dengan label `Item`.
- `i` merupakan variabel yang mewakili setiap node Item.
- `count(DISTINCT i)` menghitung jumlah node Item yang berbeda tanpa menghitung node yang sama berulang kali.
- `AS jumlah_item` memberikan nama pada kolom hasil agar menunjukkan bahwa nilai tersebut merupakan jumlah item.

➢ **Menghitung relasi interaksi**

```
MATCH (:Visitor)-[r]->(:Item)
WHERE type(r) IN ['VIEWED', 'ADDED_TO_CART', 'BOUGHT']
RETURN type(r) AS jenis_interaksi,
count(r) AS jumlah;
```

Penjelasan:

- `MATCH (:Visitor)-[r]->(:Item)` mencari relasi berarah dari node Visitor menuju node Item. Variabel `r` mewakili relasi yang menghubungkan kedua node tersebut.
- `WHERE type(r) IN [...]` membatasi hasil hanya pada tiga jenis relasi interaksi, yaitu `VIEWED`, `ADDED_TO_CART`, dan `BOUGHT`.
- `type(r)` mengambil nama tipe relasi yang ditemukan.
- `RETURN type(r) AS jenis_interaksi` menampilkan nama jenis interaksi pada kolom `jenis_interaksi`.
- `count(r) AS jumlah` menghitung banyaknya relasi untuk setiap jenis interaksi yang ditampilkan.

➢ **Memeriksa komposisi kedua jenis simpul sekaligus**

```
MATCH (n)
WHERE n:Visitor OR n:Item
RETURN labels(n) AS jenis_simpul,
 count(n) AS jumlah_simpul;
```

Penjelasan:

- `MATCH (n)` mencocokkan setiap node dalam database. Variabel `n` mewakili node yang sedang diperiksa.

- `WHERE n:Visitor OR n:Item` menyaring node agar hanya node dengan label `Visitor` atau `Item` yang diproses. Operator `OR` berarti node cukup memiliki salah satu label tersebut.
- `labels(n)` mengambil label yang dimiliki setiap node.
- `AS jenis_simpul` memberikan nama kolom untuk jenis node yang ditampilkan.
- `count(n) AS jumlah_simpul` menghitung node untuk setiap kelompok label yang dihasilkan.

- Lakukan Vector Space Modelling (VSM) terhadap simpul yang diasumsikan sebagai User dan juga Item , Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!

Jawab :
```
# Perbandingan karakteristik VSM User dan VSM Item

comparison = pd.DataFrame({
    "Karakteristik": [
        "Jumlah entitas (baris)",
        "Jumlah fitur (kolom)",
        "Jumlah pasangan interaksi nonzero",
        "Jumlah elemen matriks",
        "Kepadatan (%)"
    ],
    "VSM User": [
        vsm_user.shape[0],
        vsm_user.shape[1],
        vsm_user.nnz,
        vsm_user.shape[0] * vsm_user.shape[1],
        vsm_user.nnz / (vsm_user.shape[0] * vsm_user.shape[1]) * 100
    ],
    "VSM Item": [
        vsm_item.shape[0],
        vsm_item.shape[1],
        vsm_item.nnz,
        vsm_item.shape[0] * vsm_item.shape[1],
        vsm_item.nnz / (vsm_item.shape[0] * vsm_item.shape[1]) * 100
    ]
})

display(comparison)

print(
    "Rasio jumlah entitas User terhadap Item:",
    round(vsm_user.shape[0] / vsm_item.shape[0], 2),
    "kali"
)
```

0	Jumlah entitas (baris)	8.768600e+05	1.925810e+05
1	Jumlah fitur (kolom)	1.925810e+05	8.768600e+05
2	Jumlah pasangan interaksi nonzero	1.336552e+06	1.336552e+06
3	Jumlah elemen matriks	1.688666e+11	1.688666e+11
4	Kepadatan (%)	7.914840e-04	7.914840e-04

- Interpretasi :
  1) Jumlah entitas: VSM User memiliki 876.860 baris, sedangkan VSM Item memiliki 192.581 baris. Artinya, jumlah visitor sekitar 4,55 kali jumlah item.
  2) Jumlah fitur: VSM User memiliki fitur berupa item, sedangkan VSM Item memiliki fitur berupa visitor. Keduanya memiliki orientasi yang berlawanan.
  3) Pasangan interaksi nonzero: kedua matriks memiliki 1.336.552 elemen nonzero karena merepresentasikan pasangan interaksi yang sama.
  4) Jumlah elemen matriks: kedua matriks memiliki jumlah elemen yang sama karena transpose mengubah orientasi, bukan ukuran total matriks.
  5) Kepadatan: kepadatan keduanya sangat rendah, sekitar 0,000791484%. Ini menunjukkan bahwa sebagian besar kombinasi visitor-item tidak memiliki interaksi, sehingga format sparse lebih efisien untuk menyimpan data tersebut.

- Berdasarkan eksplorasi no. 4 dan 5, uraikan proyeksi manakah yang lebih baik (efektif/efisien)!
  Jawab :
  Berdasarkan hasil Vector Space Modelling (VSM), matriks VSM User memiliki dimensi 876.860 × 192.581, sedangkan matriks VSM Item memiliki dimensi 192.581 × 876.860. Kedua matriks mempunyai 1.336.552 pasangan interaksi nonzero dengan tingkat kepadatan sekitar 0,00079148%. Hal ini menunjukkan bahwa data interaksi bersifat sangat sparse.

  Dari sisi efisiensi, VSM Item memiliki jumlah baris sekitar 4,55 kali lebih sedikit daripada VSM User. Kondisi ini menguntungkan ketika analisis difokuskan pada pencarian item yang memiliki pola interaksi serupa. Sebaliknya, VSM User lebih sesuai apabila tujuan analisis adalah mencari pengguna dengan perilaku interaksi yang mirip.

  Pada tugas ini, proyeksi Item dipilih karena analisis berikutnya berfokus pada perhitungan cosine similarity antara satu item acuan dan item lainnya. Pemilihan ini sesuai dengan tujuan pencarian item serupa serta mengurangi jumlah entitas yang perlu dipertimbangkan dibandingkan proyeksi User. Meskipun demikian, efisiensi aktual juga dipengaruhi oleh sparsity matriks, jumlah interaksi, dan metode perhitungan similarity yang digunakan.

  Dengan demikian, proyeksi Item dinilai lebih sesuai untuk kebutuhan analisis pada tugas ini, bukan berarti selalu lebih unggul untuk semua jenis analisis graf.

- Berdasarkan analisis no. 6, lakukan perhitungan similarity pada hasil graf yg terproyeksi dengan menggu-nakan Python
  (a) Tuliskan CYPHER SQL untuk mendapatkan 5 records dengan similarity tertinggi (Anda tentukan sendirirecord acuannya)!
  Jawab :

```
MATCH (anchor:Item {itemid: 5411})-[s:SIMILAR_TO]->(other:Item)
```

```
RETURN anchor.itemid AS item_acuan,
other.itemid AS item_serupa,
s.similarity AS similarity,
s.sharedVisitors AS shared_visitors,
s.method AS metode,
s.period AS periode
ORDER BY similarity DESC
LIMIT 5;
```

(b) Tuliskan kembali nilai similarity-similarity tersebut pada graf yang telah ada! Perlihatkan hasil query-nya!
Jawab :

Item acuan	Item serupa	Cosine Similarity	Shared Visitors
5411	33030	0,137343	68
5411	33956	0,080941	22
5411	358234	0,080113	25
5411	37807	0,073341	17
5411	375492	0,063693	38

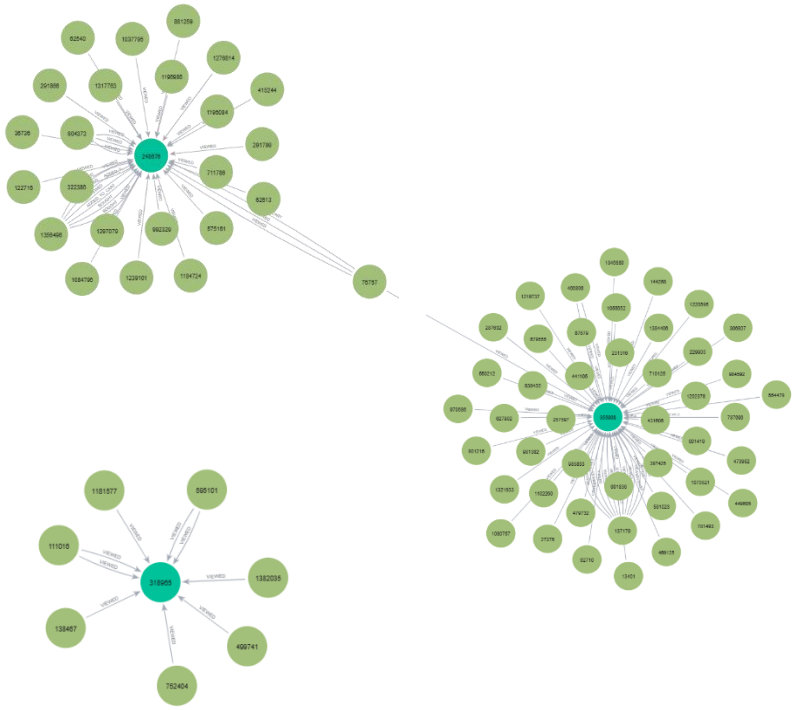
Interpretasi :
Berdasarkan hasil query, item 5411 digunakan sebagai item acuan untuk mencari lima item yang memiliki pola interaksi paling mirip. Hasil diurutkan berdasarkan nilai Cosine Similarity dari yang tertinggi hingga terendah.

Item 33030 menempati peringkat pertama dengan skor kemiripan sebesar 0,137343 dan memiliki 68 visitor yang sama dengan item acuan. Berikutnya, item 33956 memperoleh skor 0,080941 dengan 22 visitor bersama, sedangkan item 358234 memperoleh skor 0,080113 dengan 25 visitor bersama.

Item 37807 berada di peringkat keempat dengan skor 0,073341 dan 17 visitor bersama. Sementara itu, item 375492 menempati peringkat kelima dengan skor 0,063693 dan 38 visitor bersama.

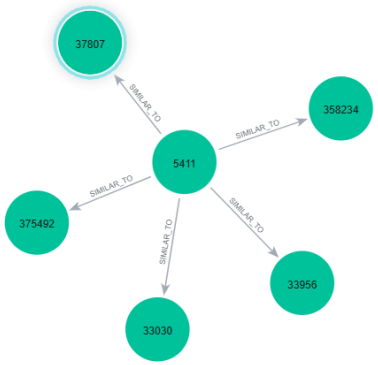
Hasil ini menunjukkan bahwa item 33030 memiliki skor kemiripan tertinggi terhadap item 5411 di antara lima item yang ditampilkan. Jumlah shared visitors tidak selalu berbanding lurus dengan skor Cosine Similarity, karena skor tersebut juga mempertimbangkan keseluruhan pola interaksi pada masing-masing item.

- Tuliskan CYPHER SQL dan tunjukkan hasil graf-nya untuk memperlihatkan kondisi: sebelum dan sesudah perhitungan similarity!
  Jawab :
  - Sebelum :
    ```
    MATCH (v:Visitor)-[r]->(i:Item)
    RETURN v, r, i
    LIMIT 100;
    ```



- Sesudah :

```
MATCH (anchor:Item {itemid: 5411})-[s:SIMILAR_TO]->(other:Item)
RETURN anchor, s, other
ORDER BY s.similarity DESC
LIMIT 5;
```



2.2 Analytics (Maks. skor: 55)

- Pilih seorang user secara acak (tentukan proses acak-nya, dan harus bisa di-reproduksi), lalu
  - tuliskan skrip Python dalam merekomendasikan KNN-nya!
    Jawab :

```
import numpy as np
```

```
import pandas as pd

# Ambil visitor yang muncul di data training dan testing
train_visitors = set(train_df["visitorid"].unique())
test_visitors = set(test_df["visitorid"].unique())

eligible_visitors = np.array(
    sorted(train_visitors.intersection(test_visitors))
)

# Pilih satu visitor secara acak dengan seed tetap
SEED = 42
rng = np.random.default_rng(SEED)

target_visitor = int(rng.choice(eligible_visitors))

# Riwayat interaksi pada P1
target_train = train_df[
    train_df["visitorid"] == target_visitor
]

# Interaksi aktual pada P2 untuk evaluasi
target_test = test_df[
    test_df["visitorid"] == target_visitor
]

print("Random seed:", SEED)
print("Jumlah visitor yang memenuhi syarat:", len(eligible_visitors))
print("Visitor terpilih:", target_visitor)
print("Jumlah interaksi P1:", len(target_train))
print("Jumlah interaksi P2:", len(target_test))
print(
    "Jumlah item unik P1:",
    target_train["itemid"].nunique()
)
print(
    "Jumlah item unik P2:",
    target_test["itemid"].nunique()
)

# Cari target yang menghasilkan minimal 3 rekomendasi
# dengan tetap menggunakan KNN k=5

from sklearn.neighbors import NearestNeighbors
import numpy as np
import pandas as pd

SEED = 42
```

```python
rng = np.random.default_rng(SEED)

# Kandidat: minimal 5 item unik di P1 dan memiliki interaksi P2
train_item_counts = train_df.groupby("visitorid")["itemid"].nunique()
test_item_counts = test_df.groupby("visitorid")["itemid"].nunique()

eligible = train_item_counts[train_item_counts >= 5].index
eligible = eligible.intersection(
    test_item_counts[test_item_counts >= 1].index
)
eligible = np.array(sorted(eligible))

# Acak urutan kandidat secara reproducible
shuffled_candidates = rng.permutation(eligible)

# Model KNN: k=5 tetangga selain target
knn = NearestNeighbors(metric="cosine", algorithm="brute")
knn.fit(interaction_matrix)

# Siapkan item per visitor agar pencarian lebih efisien
items_by_visitor = (
    train_pairs.groupby("visitorid")["itemid"]
    .agg(set)
    .to_dict()
)

target_visitor = None
recommendations = pd.DataFrame()
neighbors = []

for candidate in shuffled_candidates:
    candidate = int(candidate)
    target_idx = visitor_to_idx[candidate]

    # Ambil 6 tetangga: target sendiri + 5 tetangga lain
    distances, indices = knn.kneighbors(
        interaction_matrix[target_idx],
        n_neighbors=6
    )

    neighbors_candidate = [
        (idx, dist)
        for idx, dist in zip(indices[0], distances[0])
        if int(all_visitors[idx]) != candidate
    ][:5]

    seen = items_by_visitor.get(candidate, set())
    scores = {}
```

```
    for neighbor_idx, distance in neighbors_candidate:
        neighbor_id = int(all_visitors[neighbor_idx])
        similarity = max(0.0, 1.0 - float(distance))

        for item in items_by_visitor.get(neighbor_id, set()) - seen:
            scores[item] = scores.get(item, 0.0) + similarity

    ranked = sorted(scores.items(), key=lambda x: (-x[1], x[0]))

    if len(ranked) >= 5:
        target_visitor = candidate
        neighbors = neighbors_candidate
        recommendations = pd.DataFrame(
            ranked,
            columns=["itemid", "recommendation_score"]
        )
        break

if target_visitor is None:
    print("Belum ditemukan target dengan minimal 3 rekomendasi.")
else:
    target_train = train_df[train_df["visitorid"] == target_visitor]
    target_test = test_df[test_df["visitorid"] == target_visitor]

    print("Random seed:", SEED)
    print("Visitor target:", target_visitor)
    print("Item unik P1:", target_train["itemid"].nunique())
    print("Item unik P2:", target_test["itemid"].nunique())
    print("Jumlah tetangga:", len(neighbors))
    print("Jumlah kandidat rekomendasi:", len(recommendations))
    print("\nTiga rekomendasi teratas:")
    display(recommendations.head(3))
```

- berikan 3 item yang diiklankan untuk k = 5 pada penentuan KNN nya!

Peringkat	Item ID	Skor rekomendasi
1	351530	0,965685
2	209048	0,765148
3	100653	0,400000

- Berdasarkan hasil rekomendasi Anda no. 1b Analytics, hitunglah:
  - Hit@5
  - NDCG@5

  Jawab :

```
import numpy as np
```

```python
# Item aktual visitor pada periode testing (P2)
relevant_items = set(
    test_df.loc[
        test_df["visitorid"] == target_visitor,
        "itemid"
    ].unique()
)

# Lima rekomendasi teratas
top5 = recommendations.head(5)["itemid"].tolist()

# Relevansi setiap rekomendasi: 1 = relevan, 0 = tidak relevan
relevance = [
    int(item in relevant_items)
    for item in top5
]

# Hit@5
hit_at_5 = float(any(relevance))

# DCG@5
dcg_at_5 = sum(
    rel / np.log2(rank + 1)
    for rank, rel in enumerate(relevance, start=1)
)

# IDCG@5
ideal_relevance = [1] * min(len(relevant_items), 5)

idcg_at_5 = sum(
    rel / np.log2(rank + 1)
    for rank, rel in enumerate(ideal_relevance, start=1)
)

# NDCG@5
ndcg_at_5 = (
    dcg_at_5 / idcg_at_5
    if idcg_at_5 > 0
    else 0.0
)

print("Target visitor:", target_visitor)
print("Item relevan pada P2:", len(relevant_items))
print("Lima rekomendasi:", top5)
print("Relevansi:", relevance)
print(f"Hit@5: {hit_at_5:.4f}")
print(f"NDCG@5: {ndcg_at_5:.4f}")
```

Penjelasan :

- `relevant_items` mengambil seluruh item unik yang diinteraksikan oleh visitor `1023158` selama periode testing P2. Item ini digunakan sebagai acuan untuk mengevaluasi rekomendasi.

- `top5` mengambil lima item dengan skor rekomendasi tertinggi dari hasil KNN.

- `relevance` memeriksa setiap item rekomendasi. Nilai `1` berarti item ditemukan pada interaksi aktual P2, sedangkan `0` berarti tidak ditemukan.

- `hit_at_5` bernilai `1` jika minimal satu rekomendasi relevan ditemukan dalam lima item teratas, dan `0` jika tidak ada.

- `dcg_at_5` menghitung Discounted Cumulative Gain dengan memberikan bobot lebih tinggi pada item relevan yang berada di peringkat atas.

- `idcg_at_5` menghitung nilai DCG ideal apabila item relevan ditempatkan pada posisi teratas.

- `ndcg_at_5` membandingkan DCG aktual dengan DCG ideal untuk mendapatkan skor kualitas urutan rekomendasi yang telah dinormalisasi.

```
Target visitor: 1023158
Item relevan pada P2: 1
Lima rekomendasi: [351530, 209048, 100653, 324636, 401759]
Relevansi: [0, 0, 0, 0, 0]
Hit@5: 0.0000
NDCG@5: 0.0000
```

Anaslisis :

Berdasarkan hasil evaluasi, visitor `1023158` memiliki satu item relevan pada periode testing P2. Namun, item tersebut tidak ditemukan dalam lima item yang direkomendasikan oleh model KNN, yaitu `351530`, `209048`, `100653`, `324636`, dan `401759`.

Akibatnya, seluruh nilai relevansi rekomendasi adalah nol. Nilai Hit@5 = 0,0000 menunjukkan bahwa tidak ada item relevan yang berhasil ditemukan dalam lima rekomendasi teratas. Sementara itu, NDCG@5 = 0,0000 menunjukkan bahwa tidak terdapat item relevan dalam daftar tersebut, sehingga kualitas urutan rekomendasi berdasarkan relevansi aktual juga memperoleh skor nol.

Hasil ini tidak berarti algoritma KNN gagal untuk semua pengguna. Evaluasi ini hanya menunjukkan bahwa untuk visitor yang diuji, model belum berhasil merekomendasikan item yang diinteraksikan pada periode P2.